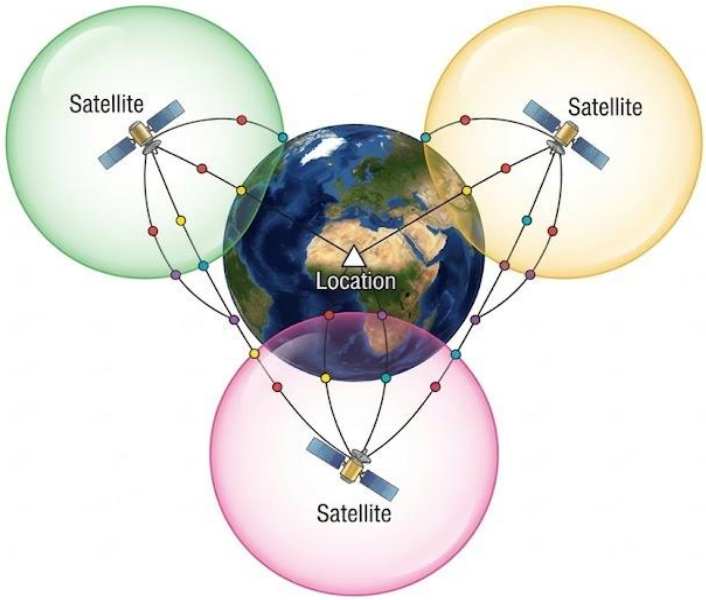
Technische Hochschule Ingolstadt

Scientific Seminar

Autonomous Vehicle Engineering

Faculty of Electrical Engineering and Computer Science



DETERMINING THE OBJECT POSITION USING GPS-LOCALIZATION

First and Last name: Aniket Harsoda

Submitted to: Prof. Dr.-Ing. Thomas Schiele

Table of Contents

LIST OF FIGURES ..... **III**

LIST OF TABLES ..... **III**

1 Introduction **1**

1.1 Project Overview 1

1.2 Importance of GPS in Modern Technology 1

2 Fundamentals of GPS Localization **2**

2.1 The Global Positioning System Constellation 2

2.2 Principles of Trilateration (How position is calculated) 2

2.3 Understanding NMEA Sentences (The data format used by GPS) 3

3 Hardware Components **5**

3.1 The NEO-6M GPS Module: Features and Specifications 5

3.2 The Arduino Mega 2560: Why Use Hardware Serial Ports? 6

4 Hardware Setup and Wiring **8**

4.1 Circuit Diagram for NEO-6M and Arduino Mega 8

4.2 Pin Mapping (VCC, GND, TX, RX) 8

5 Programming with Arduino IDE **9**

5.1 Arduino Sketch: Reading GPS Data 9

5.2 Parsing Data using TinyGPS++ Library. 10

6 Experimental Evaluation and Operational Challenges **11**

6.1 Data Analysis from the Demonstrator Setup 11

6.2 Signal Obstruction and Multipath Effects 12

6.3 Atmospheric and Satellite-Based Errors 12

6.4 Cold Start vs. Hot Start Times 13

7 Real-World Applications **13**

7.1 Consumer Navigation and Autonomous Systems (Drones and Vehicles) 13

7.2 Geofencing and Security Implementations 13

8 Enhancements and Future Work **14**

8.1 Integration with Internet of Things (IoT) platforms 14

8.2 Improving Accuracy with Differential GPS (DGPS) 14

9 Conclusion **15**

LITERATURE REFERENCES 16

LIST OF FIGURES

Figure 1: Trilateration 3
Figure 2: Circuit Diagram 8
Figure 3: Sketch and Serial Monitor 9
Figure 4: Serial Plotter 10
Figure 5: Object Position of the Experiment 102

LIST OF TABLES

Table 1: NMEA Sentences 4
Table 2: Specification of NEO-6M GPS Module 6
Table 3: Pin Mapping 8
Table 4: GPS Resolution Scales 12
Table 5: User Equivalent Range Error 12

1 Introduction

1.1 Project Overview

The construction of the described system includes two essential elements. First, there is a **u-blox NEO-6M** receiver and, secondly, **Arduino Mega 2560** microcontroller board. The latter was selected for the construction due to the number of hardware serial ports it offers. In turn, those serial ports allow processing a great amount of information from the receiver without affecting the work of the computer processor.

As far as data processing is concerned, I used a special software, namely the **TinyGPS++** library. Its role consists in converting the raw data delivered by satellites into coordinate format. Trilateration technique is applied to determine the precise geographical position of the system. To determine the accurate three-dimensional position including the altitude, it is necessary to track at least four satellites because of the necessity to calibrate the clock errors.

1.2 Importance of GPS in Modern Technology

1.2.1 Historical Context

GPS technology was originally developed by the US Department of Defense way back in the early 1960s; however, its full implementation came much later in 1995. The true breakthrough came only in May of the year 2000 when the Department of Defense deactivated the **Selective Availability (SA**) component of the GPS network. Prior to this development, GPS signals had been intentionally made less accurate for civilians so that any tracking done using such signals was limited to an accuracy level of 300 feet. Deactivation of SA resulted in the pinpoint accuracy level of 10 to 30 feet.

1.2.2 Current Technological Integration

GPS technology has today developed into a ubiquitous global utility which provides support for most of our present-day technological infrastructure systems. Some of the important uses of GPS systems are:

- **Smart Mobility**: It helps in facilitating real-time "moving maps" and routing of cars and cabs.
- **Industrial Automation**: It facilitates efficient automation technologies for sectors such as aviation, industrial logistics, and agricultural automation technologies.
- **Public Safety and Security**: It is used for purposes including lifesaving technologies like E911 cell phone tracking, which helps in locating people in trouble in emergencies.
- **Consumer Wearables**: GPS technology has helped design personal safety products, e.g. GPS enabled watches to find missing person. *[1]*

2 Fundamentals of GPS Localization

2.1 The Global Positioning System Constellation

This system employs anywhere between **24 and 31 satellites** positioned along six orbits. Each orbit is named from A to F. All orbits have an inclination of **55°** to the equator, located at **20,200 km** in altitude. Such positioning guarantees coverage of the entire world, with each satellite completing **two** full revolutions every **24 hours**.

Relativistic Precision and Atomic Clocks
Each satellite relies on both Cesium and Rubidium atomic clocks to maintain the most precise timing to a level of nanoseconds. Nevertheless, to avoid enormous positioning errors, this system needs to account for the effects described by Albert Einstein:

- **Time Drift:** As a result of the interaction between gravity and velocity (Special and General Relativity) the clocks of the satellites would operate fast by **38 µs** each day.
- **Correction:** The resulting positioning error would amount to **10 Km** per day without corrections.
- **Frequency Offset:** Therefore, the engineers compensate by tuning the clock frequencies by **10.22999999543 MHz** in advance. Thus, once the signal is transmitted, it is received precisely as **10.23 MHz.** [2]

2.2 Principles of Trilateration (How position is calculated)

Trilateration is the basic concept on which GPS operates, as it involves determining the position by calculating distances. GPS is, however, a **time of arrival (TOA)** system. What makes this technology exciting is the great deal of precision that goes into the calculations.

The High-Speed Challenge
Because GPS radio signals travel at the speed of light, roughly $3 \times 10^8\ m/s$, timing is everything.

- **The Error Factor:** If there is a $\mathbf{1}\boldsymbol{\mu s}$ synchronization error between the satellite and your receiver, this translates into a huge positional error of **300 meters**.
- **The Math:** To figure out your location, the system must solve a set of quadratic equations to find the intersection of the signal ranges:

$$\sqrt{((x_k - x)^2 + (y_k - y)^2)} = r_k$$

In this equation, (x_k, y_k) is the known satellite position, (x, y) is the unknown user position, and r_k is the measured distance.

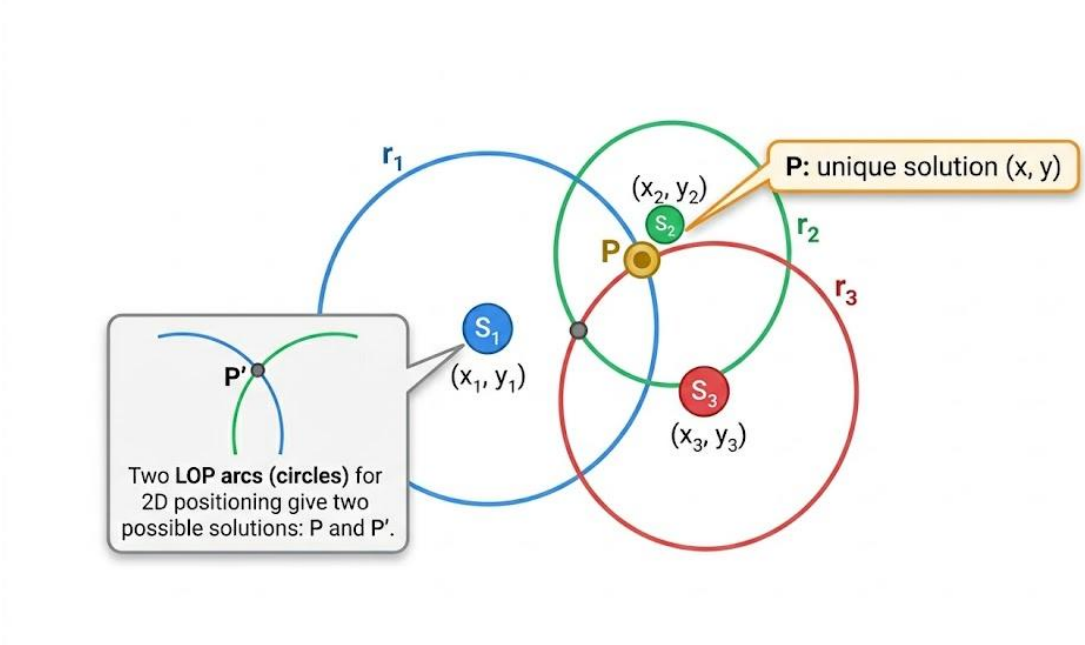


Figure 1: Trilateration

The "Fourth Satellite" Secret

One interesting query about the Global Positioning System (GPS) is why GPS systems require four satellites to determine their position in 3D space when only three would suffice geometrically?

- **Keeping Sensors Cheap:** To make devices like the NEO-6M cost-effective and compact, atomic clocks are avoided.
- **The Mathematical Solution:** In lieu of using a flawless clock, the GPS system relies on mathematical solutions that minimize requirements for the clock on the receiver. This requires the use of an **additional radio frequency carrier**, i.e., the fourth satellite.
- **Timing Error Elimination:** By employing the fourth signal, the difference in time, i.e., clock bias, is mathematically eliminated. *[3]*

2.3 Understanding NMEA Sentences (The data format used by GPS)

The NMEA protocol is basically a set of standards through which communication between different electronic equipment such as GPS and computer can take place. In IEEE 0183 format, GPS messages are transmitted on a line basis known as **sentences** with each sentence being an entire and complete message. **Technical Rules of NMEA Sentences:**

- **Start and End:** Every sentence must begin with the **$** character and end with a carriage return and line feed ().
- **Character Limit:** A standard sentence cannot exceed **80 characters** of visible text.
- **Data Separation:** Individual data items within a sentence are separated by **commas**.
- **The Checksum:** Each sentence contains a checksum for data validation. This is an **8-bit exclusive OR (XOR)** of all characters between the '$' and '*' characters.

Key NMEA Sentences for Localization While the NMEA standard defines many sentences, the following are the most critical for determining an object's position:

Sentence	Description	Key Information Provided
$GPGGA	Global Positioning System Fix Data	Time, position, fix quality, number of satellites, and **altitude.**
$GPRMC	Recommended Minimum Specific GPS Data	Time, date, position, and **ground speed**. The most standard "positioning" sentence.
$GPGSA	GPS DOP and Active Satellites	Details on the satellites used for the fix and **Dilution of Precision (DOP)** values.
$GPGSV	GPS Satellites in View	Number of satellites in view, their elevation, azimuth, and **signal strength (SNR).**
$GPVTG	Track Made Good and Ground Speed	Specifically provides actual track and speed over the ground.

Table 1: NMEA Sentences [4,5]

3 Hardware Components

3.1 The NEO-6M GPS Module: Features and Specifications

The **u-blox NE0-6M** is a good GPS module produced by a Swiss company. The module employs u-blox's advanced sixth generation u-blox 6 positioning system. As mentioned above, the device is relatively small and easy to connect. Therefore, it is widely used in many different applications for battery-operated autonomous devices (like drones).

Build and Signal Protection:
The GPS Module itself is a printed circuit board. One can see a distinctive **Ceramic Patch Antenna**, which, according to u-blox specifications, is tuned to catch the standard GPS frequency of 1575.42 MHz. Moreover, the main processing chip (which is a GPS chip) is located inside an electromagnetic Faraday cage, which protects sensitive elements from the electronic noise. The cage is essential because the GPS module uses sophisticated anti-jamming technologies that guarantee a stable connection with satellites despite the high level of interference.

Specialized Technologies:
There are several features that make the u-blox NEO-6M unique because they help in finding a user's location fast and precisely:

- **50-Channel Positioning Engine**: This powerful component can be viewed as a kind of high-speed satellite search engine that utilizes over a million components to search 50 GPS channels simultaneously. Thus, it finds a signal in under a second during Hot Start.
- **SuperSense® Technology**: The module tracks exceptionally weak signals that are critical to maintaining a GPS connection in tree canopies or partially covered areas.
- **AssistNow™ Support**: It is an "Assisted GPS" technology that helps the module to start the signal tracking faster by receiving satellite paths pre-calculated in another computer.
- **Dual Communication Protocols**: The NEO-6M "speaks" two languages:
    - **NMEA 0183 Protocol**: The default global standard for all humans' readable messages. (discussed in Chapter 2).
    - **UBX Binary Protocol**: A high-efficiency, binary-only language used for faster data transfer and deep configuration changes.
- **Built-in Memory & Battery**: The module has its own built-in memory **(EEPROM)** and a rechargeable backup battery. These two work together to save the most recent satellite data even when you turn the power off. This allows the module to perform a "Warm Start," meaning it can find your location much faster the next time you turn it back on. [5]

Parameter	Specification
Chipset	u-blox 6 positioning engine
Receiver Type	50-channel GPS L1 C/A code
Horizontal Position Accuracy	2.5 m (CEP) / 2.0 m (with WAAS)
Maximum Navigation Update Rate	5 Hz
Tracking Sensitivity	-161 dBm to -162 dBm
Cold Start Time	26 - 27 seconds
Hot Start Time	< 1 second
Operating Voltage	2.7 V to 3.6 V (Module input often 3.0 V - 5.0 V)
Communication Interfaces	UART, USB, SPI, DDC (I2C)
Baud Rate	4800 to 230400 bps (Default: 9600 bps)
Reference Coordinate System	WGS-84
Operating Temperature	-40 °C to +85 °C

Table 2: Specification [5]

3.2 The Arduino Mega 2560: Why Use Hardware Serial Ports?

This GPS project utilized the Arduino Mega 2560 due to its distinctive communication scheme. Unlike basic Arduino boards, which consist of one hardware serial port, the Mega 2560 has **four distinct hardware serial ports**, or UARTs, known as Serial, Serial1, Serial2, and Serial3.

Why the Mega 2560 Communication Matters:

- **Hardware Serial Port Efficiency:** It means that the hardware serial port is nothing but an actual circuit inside the ATmega2560 microcontroller. In addition, the hardware serial works like a little additional processor which handles data synchronization and organization on its own, so the CPU does not have to deal with it anymore.
- **Background Buffering:** Such ports also have their own internal buffer memory where the incoming GPS data is saved automatically in the background. As a result, there will be no data loss in case the CPU becomes engaged in some other activities, which would be

impossible with software serial due to the necessity of pausing all the processes and monitoring incoming data one-by-one.

- **System Parallelism:** The Mega functions as a central hub for multiple devices at once. It can communicate with the NEO-6M module, a cellular module for remote tracking, and a computer for **debugging** simultaneously without any signal overlaps or conflicts.
- **Baud Rate Stability:** GPS receivers usually transfer a lot of information very quickly, i.e., at such baud rates as 115,200 bps. While the hardware serial port maintains a stable connection at those rates, a software version will become unstable after some time. *[6]*

4 Hardware Setup and Wiring

4.1 Circuit Diagram for NEO-6M and Arduino Mega

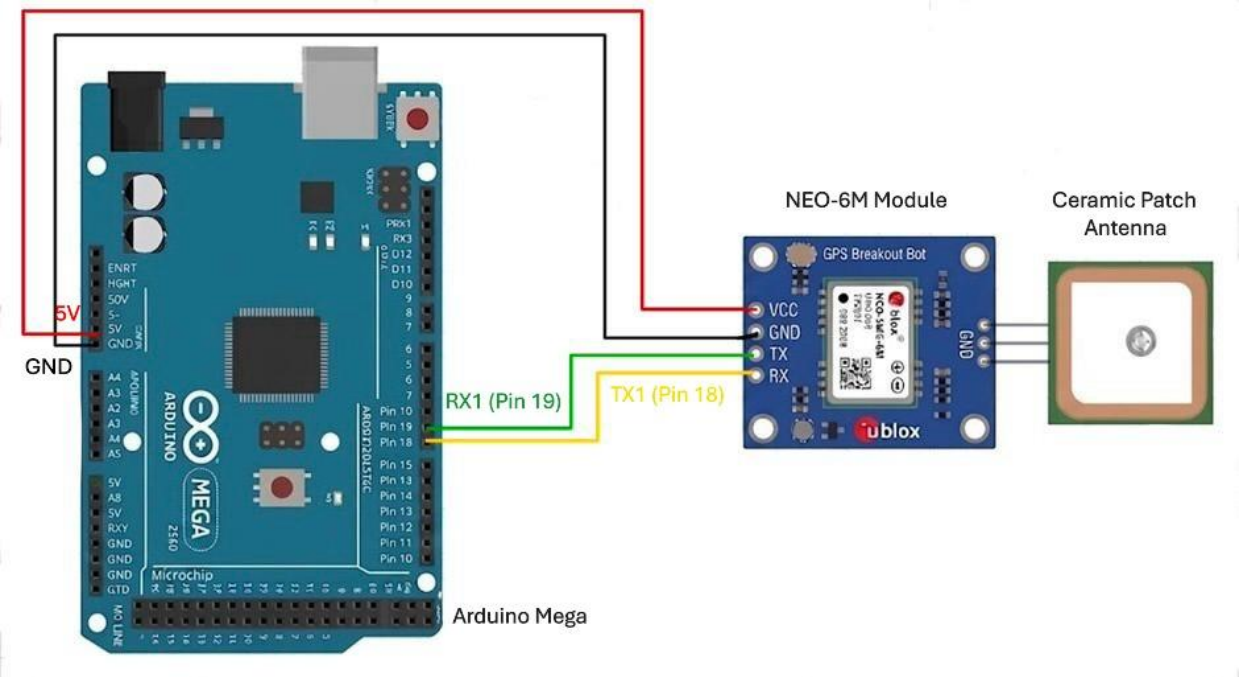


Figure 2: Circuit Diagram

4.2 Pin Mapping (VCC, GND, TX, RX)

	NEO-6M GPS Pin	Arduino Mega 2560 Pin	Function Description	Connection Note (Important!)
	VCC	5V	Power Supply	Connection verified in user setup.
	GND	GND	System Ground	Connection verified in user setup.
	TX	Pin 19 (RX1)	Transmit Data	3.3V GPS TX connects directly to 5V Mega RX (Safe).
	RX	Pin 18 (TX1)	Receive Data	REQUIRES 5V to 3.3V LEVEL SHIFT.

Table 3: Pin Mapping

5 Programming with Arduino IDE

5.1 Arduino Sketch: Reading GPS Data

GPS_NEO_6M.ino

```ino
#include <TinyGPS++.h>

// Create a GPS parser object from the TinyGPS++ library to decode raw satellite sentences.
TinyGPSPlus gps;

void setup() {
  // Initialize the main serial port for communication with the PC (Serial Monitor/Plotter).
  Serial.begin(9600);

  // Initialize Hardware Serial 1 (Pins 18 and 19) to receive incoming data from the NEO-6M module.
  Serial1.begin(9600);
}

void loop() {
  // Continuous check: Keep reading data as long as incoming bytes exist in the hardware buffer.
  while (Serial1.available() > 0) {

    // Feed raw data characters into the encoder. Returns true when a complete NMEA sentence is parsed.
    if (gps.encode(Serial1.read())) {
      displayParsedData(); // Trigger our custom function to display the processed data
    }
  }
}

void displayParsedData() {
  // Safety Gate: Only print data if the module has resolved clock bias and established a stable 3D fix.
  // This prevents the system from cluttering your screen with blank or "zero" data during startup.
  if (gps.location.isValid()) {

    // Note: The F() macro keeps static text strings inside Flash memory to save limited dynamic RAM (SRAM).
    // Commas are carefully printed between values to allow the Serial Plotter to split and graph multiple lines cleanly.

    Serial.print(F("Latitude:"));
    Serial.print(gps.location.lat(), 6); // Print latitude to 6 decimal places for precise high-resolution positioning.
    Serial.print(F(","));

    Serial.print(F(" Longitude:"));
    Serial.print(gps.location.lng(), 6); // Print longitude to 6 decimal places for precise high-resolution positioning.
    Serial.print(F(","));

    Serial.print(F(" Satellites:"));
    Serial.print(gps.satellites.value()); // Print active satellite count; a minimum of 4 is required for a 3D fix.
    Serial.print(F(","));

    Serial.print(F(" Altitude:"));
    Serial.println(gps.altitude.meters()); // Print altitude in meters; println terminates the data row packet cleanly.
  }
}
```

Output | Serial Monitor

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM4')

```
16:23:23.540 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.10
16:23:23.605 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.10
16:23:23.703 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.10
16:23:24.248 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.10
16:23:24.316 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.10
16:23:24.381 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:24.480 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:24.550 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:24.609 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:24.674 -> Latitude:48.767002, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:25.272 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:25.337 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.20
16:23:25.403 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.00
16:23:25.468 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.00
16:23:25.567 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.00
16:23:25.632 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.00
16:23:25.698 -> Latitude:48.767005, Longitude:11.433200, Satellites:6, Altitude:375.00
```

Figure 3: Sketch and Serial Monitor

The picture below depicts the execution of the code on the Arduino Mega 2560 board employing the **TinyGPS++ library**. The program receives NMEA data from the NEO-6M via the hardware serial communication protocol. The **Serial Monitor** displays the successful reception with 3D position lock and **6 satellites** providing exact 6 decimal latitude and longitude values.

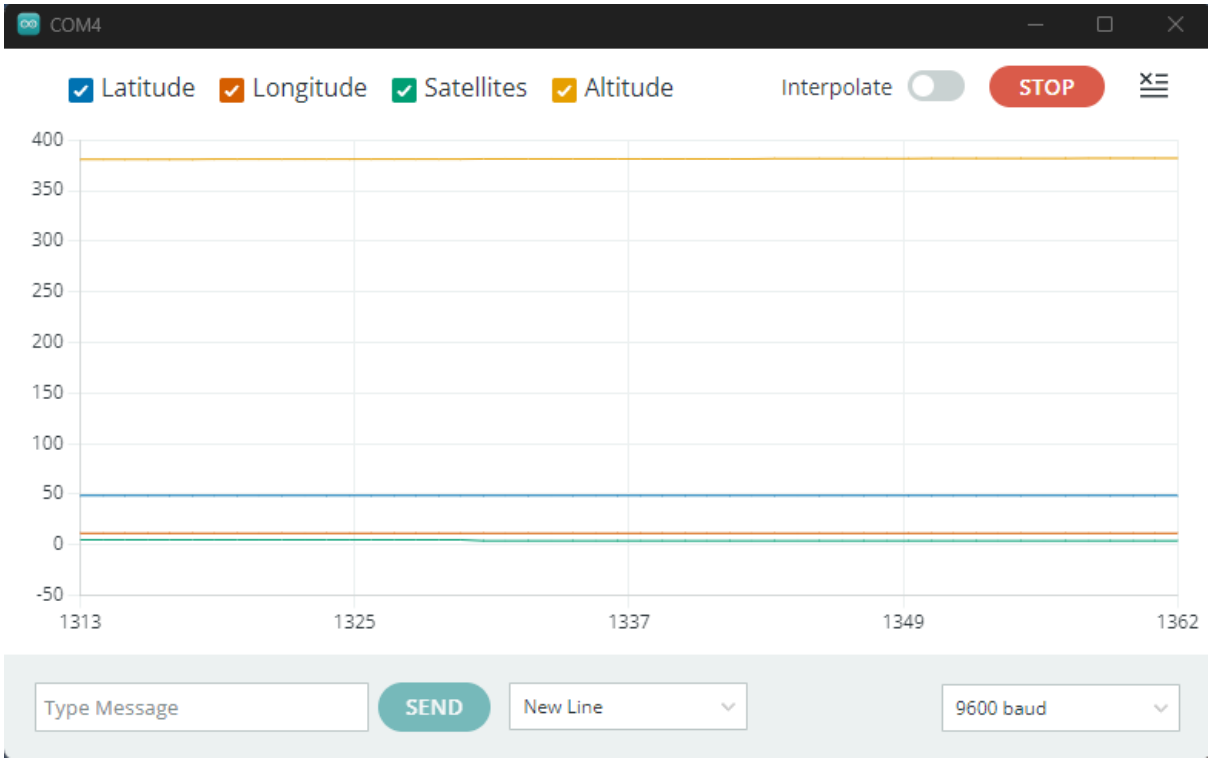


Figure 4: Serial Plotter

The GPS signal stability can be represented in the form of a chart below by means of the Arduino **Serial Plotter**. Since the output of the signal is displayed in the form of a series of numbers with relevant labels, you can see how latitude, longitude, number of satellites, and altitude change on the chart. The straight line on the chart proves good lock.

5.2 Parsing Data using TinyGPS++ Library.

- **Decoding with Object-Oriented NMEA:** The firmware employs the **TinyGPS++** library in decoding the raw sentences from the module. Rather than having cumbersome manual coding processes, the gps.encode() function decodes the sentences accurately character by character to form an elegant data flow.
- **Data Integrity and High-Precision Extraction:** To ensure high data integrity, the extraction process ensures that data is extracted up to six decimal places of precision. Significant .isValid() validation process works like a safety wall where only valid data is presented while the "zero" data resulting from warm-up of the module is ignored.
- **Dual-Mode Visualization and Monitoring:** The final data format becomes simple with labelling in place. With the proper data format, there will be two methods of processing the data; viewing the data on Serial Monitor for precise coordinates and viewing the data on Serial Plotter for monitoring purposes. [7]

6 Experimental Evaluation and Operational Challenges

6.1 Data Analysis from the Demonstrator Setup

The prototype that incorporates the u-blox NEO-6M and Arduino Mega 2560 was put to the test right here in the Technische Hochschule Ingolstadt campus near D building. With the aid of a stable 3D fix with **six** satellite connections, the following location coordinates were acquired:

- **Latitude:** 48.767002° N
- **Longitude:** 11.433200° E
- **Altitude:** 375.10 m

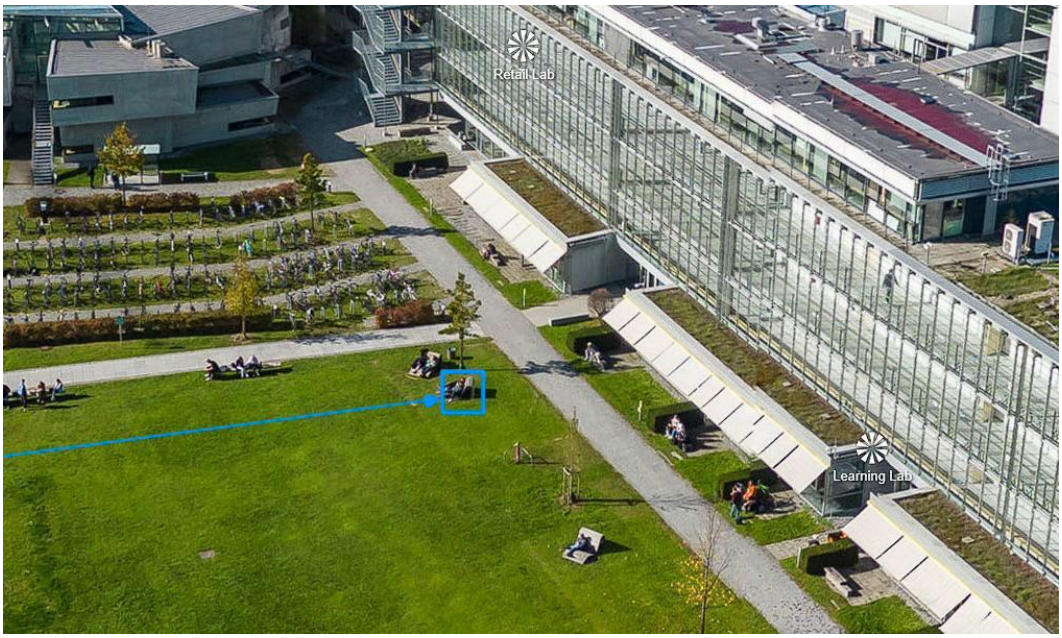


Figure 5: Object Position of the Experiment

Mapping the recorded coordinates confirms the experimental location within an approximate **2.5 m margin of error**. While the firmware displays data at a precision of six decimal places (11.1 cm resolution), the true physical accuracy remains bounded by the module's 2.5 m Circular Error Probable (CEP)**.**

Precision	Resolution	Physical Interpretation
4 Decimals	0.0001° ≈ 11.1 m	Standard vehicle navigation
5 Decimals	0.00001° ≈ 1.11 m	High-fidelity pedestrian navigation
6 Decimals	0.000001° ≈ 11.1 cm	Precision geodetic surveying

Table 4: GPS Resolution Scales [8]

6.2 Signal Obstruction and Multipath Effects

Tests conducted around the historical center of Ingolstadt have the phenomenon of urban canyons, where high buildings hinder the transmission of signals by **20-30 dB**, which usually brings the power of the signal under the detection threshold.

To determine a position, the module solves trilateration equations using the known satellite coordinates $(\mathrm{x_k}, \mathrm{y_k}, \mathrm{z_k})$ and the measured distances, or pseudoranges r_k :

$$(x_k - x)^2 + (\mathrm{y_k} - y)^2 + (z_k - z)^2 = r_k^2$$

As all our monitored satellites are placed above the surface in the sky, the **Vertical Dilution of Precision (VDOP)** value is going to be significantly higher in comparison to the Horizontal Dilution of Precision. Such geometry becomes the reason for the sudden jump of the height value from 10 to 100 meters in case there is only 4-5 satellites' weak signal connection. *[9]*

6.3 Atmospheric and Satellite-Based Errors

Deterministic factors contribute to the **User Equivalent Range Error (UERE)**. The primary sources and mitigation techniques include:

Error Source	RMS Error (m)	Primary Mitigation
Ionospheric Delay	5.0	Klobuchar model
Orbital/Clock Error	2.5	Ephemeris updates
Tropospheric Delay	0.5	Hydrostatic models
Multipath/Noise	1.0	Antenna Ground plane

Table 5: User Equivalent Range Error *[10]*

- **Ionospheric Refraction:** Particles in the atmosphere cause signals to travel slower through them. The correction for this error is made by applying the Klobuchar model, which takes care of roughly 70% of this error.
- **Relativistic Correction:** time dilation causes satellite atomic clocks to run faster, specifically by **38 μs/day**. This is pre-corrected to avoid a drift of up to 10 km per day.

6.4 Cold Start vs. Hot Start Times

Time To First Fix (TTFF) is dependent on the orbital data held by the battery backed SRAM of the receiver:

- **Cold Start (27–50s):** Memory is empty It will need a fresh **Almanac and Ephemeris** download (27-50 seconds).
- **Warm Start (≈25s):** Valid almanac exists, but new ephemeris download needed.
- **Hot Start (< 1s):** Valid orbital data exists. Localization is practically instant.

Hardware Challenge: The breakout board makes use of the **MS621FE** as its backup battery. If this battery is dead, the module will always be in a "cold start" mode during every test period. [9]

7 Real-World Applications

7.1 Consumer Navigation and Autonomous Systems (Drones and Vehicles)

With tiny dimensions, the u-blox NEO-6M module revolutionized two sectors:

- **Autonomous vehicle navigation and tracking:** In this application, the module serves as the primary sensor node. It communicates standard NMEA messages to a processor, such as the Arduino Mega 2560, for tracking its position, velocity, and direction in real-time. The information helps find an optimal route and maintain the integrity of cargo.

- **Autonomous UAV Navigation (Drones):** In drones with flight control software, including **Betaflight and PX4**, constant connectivity is required for **"Return-to-Home" (RTH)** features. In the absence of a GPS signal, the drone uses saved home coordinates for flying back. Although the 1 Hz update rate is acceptable for recovery flights, faster autonomous flights may require more advanced engines running on 10 Hz, including M10 models. *[11]*

7.2 Geofencing and Security Implementations

In geofencing, virtual borders are set to activate a certain action, such as an alert about theft or speed limits once the border is crossed. To minimize latency while reducing costs, the calculations occur through **Edge Computing**, which means that the algorithm executes on-site, directly on a microprocessor.

For a circular geofence with center (ϕ_1, λ_1) and radius r, the distance d to the current position (ϕ_2, λ_2) is calculated using the **Haversine Formula:**

$$d = 2R.arcsin\left(\sqrt{sin^2\left(\frac{\Delta\phi}{2}\right) + cos(\phi_2)cos(\phi_2)\sin^2\left(\frac{\Delta\phi}{2}\right)}\right)$$

Where $\mathrm{R} = 6371\ \mathrm{km}$. On 8-bit architectures like the ATmega2560, where trigonometric operations are computationally expensive, the **Equirectangular Approximation** is often used for efficiency:

$$d = R\sqrt{((\Delta\phi)^2 + (cos(\phi_m)\Delta\lambda)^2)} \qquad [12]$$

Edge-Based Stabilization Algorithm:

To prevent **"Boundary Chatter"**—false entry or exit triggers caused by GPS drift near the perimeter—two software filters must be implemented:

- **Hysteresis:** Two different radii can be used for entering and exiting, such as 100 m for entering and 115 m for exiting.

- **Debouncing:** A minimum number of GPS sample readings, say $N = 5$, should be observed on the other side of the geofence border before deciding. *[13]*

8 Enhancements and Future Work

8.1 Integration with Internet of Things (IoT) platforms

Making this prototype into a cloud-based IoT unit entails connecting it with the cellular **SIM800L GPRS** module with the help of the Arduino Mega board. It is optimal for implementing cellular connectivity due to its hardware serial ports. The Arduino Mega can receive GPS data via Serial1 port and send commands to the cellular module via Serial2. In this way, it will become possible to transfer coordinate data using JSON format to IoT platforms like **MathWorks ThingSpeak**.

- **Critical Power Supply Challenges:** The SIM800L GPRS cellular module needs bursts of high current that reach up to **2 A**. Directly powering the cellular module with an Arduino Mega may result in a voltage drop leading to a complete system crash or restart. To avoid any failures, it is necessary to provide external **4.0V** power supply along with a capacitor of **1000 µF** between power supply pins to ensure sufficient stability of the electric circuit. *[11]*

8.2 Improving Accuracy with Differential GPS (DGPS)

Ordinary accuracy of standard GPS receivers ranges from several tens to several dozen meters (for NEO-6M, it is approximately 2.5 meters). Nonetheless, there are two ways in which this result can be improved significantly:

- **Satellite-Based Augmentation Systems (SBAS):** NEO-6M already includes SBAS functionality (like WAAS or EGNOS). This means that data transmitted by ground stations can be received through the satellite. As a result, the accuracy of the system increases to roughly **2.0 meters** due to reduced effects of atmospheric and orbital delays.

- **Ground-Based DGPS and RTCM 2.3:** It ensures the receiver will be capable of eliminating all common mode errors, providing accuracy of **1.0 – 1.5 meters**.

Future Improvements: It should be noted that a NEO-6M receiver does not allow achieving centimeter level accuracy due to its inability to carry out carrier phase measurements. Thus, for further developments in the same area at TH Ingolstadt, it is recommended to move to u-blox ZED-F9P, which provides millimeter accuracy**.** *[11]*

9 Conclusion

In summary, the implementation proved the viability of constructing highly precise tracking solutions through combining the u-blox NEO-6M receiver together with the Arduino Mega 2560 board. The choice of the receiver itself proved to be wise in terms of meeting our requirements regarding the "User Segment." Namely, it had a powerful 50-channel engine complemented by the Faraday Cage shielding that kept the received signal unaffected even amid a lot of electromagnetic pollution. In turn, throughout all experiments, I could ascertain the crucial importance of the Arduino's dedicated hardware UARTs. They provided an effective processing of huge amounts of NMEA data without overloading the CPU.

I managed to achieve a stable three-dimensional fix utilizing six satellites with coordinates **48.767002°N, 11.433200°E** and **375.10 m** above sea level. Despite my code having the ability to calculate positions to six decimals of a degree (approximately 11.1 cm), the physical accuracy was still determined by the 2.5 meters Circular Error Probable (CEP) of the module. Moreover, due to the presence of "urban canyon" effect in the surroundings of the TH Ingolstadt, I experienced minor changes in altitude in accordance with VDOP; however, no issues occurred with the horizontal positions.

What should also be considered is the critical role of the backup battery in achieving optimal performance of the system. Indeed, in case it runs out of power, the "Hot Start" cannot occur; the receiver would always have to go through the 27-50 seconds long "Cold Start."

Overall, the implementation has shown its capability of serving in practical tasks such as "Return-to-Home" protocols for drones and other applications of edge-based geofencing. Utilizing the formulas including Haversine and Equirectangular Approximation, I could prove that even 8-bit processors can process complex boundaries adequately.

Finally, speaking about future work, we would need to migrate from the NEO-6M towards more advanced dual-frequency RTK receivers such as u-blox ZED-F9P to conduct further testing in the context of automotive applications.

LITERATURE REFERENCES

1. *Broida, Rick. How to do everything with your GPS, 2003. McGraw-Hill/Osborne. And* URL https://www.geotab.com/blog/what-is-gps/
2. *Kaplan, E. D., & Hegarty, C. J. (Eds.). (2006). Understanding GPS: Principles and Applications (2nd ed.). Artech House. ISBN: 1-58053-894-0*
3. *Misra, Pratap and Enge, Per. Global Positioning System: Signals, Measurements, and Performance, 2nd Edition, 2006. Ganga-Jamuna Press. ISBN: 978-0970954428*
4. *https://content.u-blox.com/sites/default/files/products/documents/u-blox6_ReceiverDescrProtSpec_%28GPS.G6-SW-10018%29_Public.pdf*
5. URL https://ieeexplore.ieee.org/document/6514210 *Shoab, M., Jain, K., Haq, M. A., & Shashi, M. (2013). "Development and implementation of NMEA interpreter for real time GPS data logging." 2013 International Conference on Communication and Signal Processing (ICCSP). IEEE.*
6. URL https://www.alldatasheet.com/datasheet-pdf/pdf/1283987/U-BLOX/NEO-6M.html?utm_source
7. Hart, M. (2013). *TinyGPS++: A new, full-featured GPS parser for Arduino*. Retrieved from http://arduiniana.org/libraries/tinygpsplus/
8. URL https://www.whitelabeltracking.com/how-many-satellites-get-a-good-gps-lock-and-why/
9. *u-blox. NEO-6 Series Technical Datasheet. Document GPS.G6-HW-09005* URL https://content.u-blox.com/sites/default/files/products/documents/NEO-6_ProductSummary_%28GPS.G6-HW-09003%29.pdf
10. *VectorNav Technologies. "GNSS Error Budget and Atmospheric Propagation Delay Primer* URL https://www.vectornav.com/resources/inertial-navigation-primer/specifications--and--error-budgets/specs-gnsserrorbudget
11. *Last Minute Engineers. "NEO-6M GPS Module Interfacing Tutorial with Arduino*. URL https://www.waveshare.com/wiki/UART_GPS_NEO-6M
12. *Usman, A. O. (2026). "Design and Implementation of a Low-Cost IoT-Based Vehicle Tracking System Using ESP32 and GPS Technology." International Journal of Science, Engineering and Technology* URL https://lastminuteengineers.com/neo6m-gps-arduino-tutorial/
13. *Almuhaiza, J. K. I. S. (2025). Edge IoT Geofencing for Asset Tracking using Low-Power Microcontrollers*. URL https://pcbsync.com/gps-antenna-pcb/